

深層強化学習を用いた柔軟構造衛星のロバスト姿勢制御系設計

○遠藤亮 山口功 山崎武志 高野博行 (防衛大学校)

Deep Reinforcement Learning Algorithm on Robust Attitude Control System Design for a Flexible Space Structure

*R. Endo, I. Yamaguchi, T. Yamasaki and H. Takano (NDA)

Abstract: This paper describes the applications of deep reinforcement learning for the design of robust flexible spacecraft attitude control systems. For comparison, we design attitude control systems using a typical H-infinity mixed sensitivity problem and a fixed controller structure problem. Numerical simulations are carried out to demonstrate the validity of the proposed method.

Keywords: Deep reinforcement algorithm, Robust spacecraft attitude control system

1 はじめに

機械学習により設計されたコントローラによる制御・誘導が宇宙工学の分野においても注目を浴びている。本稿は機械学習の一種である強化学習を用いて、柔軟構造衛星の姿勢制御系を設計するとともに、ロバスト制御と比較することで宇宙機の姿勢制御への応用について検討する。

1.1 背景と目的

強化学習は一連の行動について報酬を与え、その報酬を最大化するように設計された学習モデルである。もともとは動物の行動学習のモデルとしてはじまった分野であるが、工学分野においても高い応用性を有していることが知られており、高度な制御理論を駆使せずとも複雑・未知なプラントのコントローラを設計できるという長所がある。また、他の機械学習と異なり、学習に必要なビックデータを用意する必要がなく、自分自身で試行錯誤を繰り返しながら学習する点においても大きな長所と言える。宇宙機の姿勢制御は、諸元や擾乱、モデル化誤差など、ミッションにより与えられる環境が大きく異なり、他のミッションのビックデータから学習できることは限定的である。その点、強化学習は外部からの知識を一切与えずに運動のパターンを学習することができる。

本稿では宇宙機の姿勢制御系設計を考えるにあたり、強化学習による適応例があまりみられない柔軟構造衛星を想定した。大型の構造物を有する宇宙機の制御モデルにおいては弾性変形が及ぼす動的な影響を無視することはできない。この際、宇宙機を剛体ではなく柔軟構造物として扱う必要がある。車両や航空機と異なり、無重量状態での運用を想定している宇宙機は地上試験等の機会が限られており、モデル化の誤差が姿勢制御に与える影響が大きい。モデル化誤差をもつ柔軟

構造衛星の姿勢制御においては、ETS-VI (きく6号) などをはじめとする多数の宇宙機がロバスト制御を採用している。本稿ではロバスト制御による柔軟構造衛星の制御系を設計し、ニューラルネットワークと動的計画法の融合により大きな進展をみせた学習方式である深層強化学習 (Deep Reinforcement Learning) により設計した制御系と比較・検討を行う。

2 柔軟構造衛星の1軸数学モデル

本稿で扱う柔軟構造衛星の線形化モデルを記述する。衛星は軌道上で3次元6自由度運動をするが、ここでは簡単のため、姿勢1軸回りの運動方程式を考え、それを状態空間で表現すると、

$$\dot{\mathbf{x}}_F = \mathbf{A}_F \mathbf{x}_F + \mathbf{B}_F u \quad (1)$$

$$\mathbf{y} = \mathbf{C}_F \mathbf{x}_F \quad (2)$$

ここで状態量 $\mathbf{x}_F$ は非拘束モードモデル [1] におけるモード変数で構成され、

$$\mathbf{x}_F = [\boldsymbol{\xi}^T \quad \dot{\boldsymbol{\xi}}^T]^T \quad (3)$$

となる。もし、すべてのモードを考慮して、このモデルを構築する場合は、これを「フルモデル」と呼ぶ。下付き F 文字はフルモデルを意味している。

さて、このモード変数はスカラー剛体モード (ξ_r) とベクトル弾性振動モード (ξ_f) を表すモード変数に分けることができる。

$$\boldsymbol{\xi} = [\xi_r^T \quad \xi_f^T]^T \quad (4)$$

弾性振動モードを m 次とすると、 $\boldsymbol{\xi}$ は $(m+1)$ 次、状態量 $\mathbf{x}$ は $n = 2(m+1)$ 次となる。また、状態方程式の各行列は $\mathbf{A}_F$ が $n \times n$ 次正方、 $\mathbf{B}_F$ が $n \times 1$ 次縦行列であり、それぞれ

$$\mathbf{A}_F = \begin{bmatrix} \mathbf{O} & \mathbf{U}_{(m+1) \times (m+1)} \\ -2Z\Omega & \Omega^2 \end{bmatrix}, \quad \mathbf{B}_F = \begin{bmatrix} \mathbf{O} \\ \Phi^T \end{bmatrix} \quad (5)$$

と表せる．ここで， Z はモード減衰比行列， Ω は固有角周波数行列， Φ はモード形状行列と呼ばれ，それぞれ

$$Z = \text{diag}[0, \zeta_1, \zeta_2, \dots, \zeta_m] \quad (6)$$

$$\Omega = \text{diag}[0, \omega_1, \omega_2, \dots, \omega_m] \quad (7)$$

$$\Phi = [\phi_0, \phi_1, \dots, \phi_m] \quad (8)$$

と定義できる．一方，観測方程式については，例えば姿勢角と姿勢角速度の2要素が検出できる場合は， C_F 行列は $2 \times n$ 次の横長行列になり

$$C_F = \begin{bmatrix} \Phi & O \\ O & \Phi \end{bmatrix} \quad (9)$$

となる．これらの結果から，フルモデルでの u から $y_1 = \theta$ (ロール姿勢角) までの伝達関数 $G_{F1}(s)$ は

$$G_{F1} = \frac{\phi_0^2}{s^2} + \sum_{i=1}^m \frac{\phi_i^2}{s^2 + 2\zeta_i\omega_i + \omega_i^2} \quad (10)$$

となる．

次に姿勢制御系を設計する際に基準となるモデルは，このようなフルモデルではなく，これを低次元した制御対象モデルを用いることが一般的である．混合感度問題では制御対象モデルと同次数の制御器が設計されるため，対象となるモデルの次数はできるだけ低次元であることが望まれる．また衛星実機に姿勢制御プログラムを実装する際でも，搭載機器のCPUメモリの制約から，低次元の制御器が要求される傾向がある．ここでも制御対象モデルとして，(a) 剛体のみ，あるいは (b) 剛体+最低次振動モードのみを取り込んだ低次元化モデルを「ノミナルモデル」と称する．例えば，(b) の場合は，状態/観測方程式は

$$\dot{\mathbf{x}}_N = A_N \mathbf{x}_N + B_N u \quad (11)$$

$$\mathbf{y} = C_N \mathbf{x}_N \quad (12)$$

ここで状態量 $\mathbf{x}_N$ は，剛体モードを ξ_0 ，1次振動モード ξ_1 をとすると

$$\mathbf{x}_N = [\xi_0 \quad \xi_1 \quad \dot{\xi}_0 \quad \dot{\xi}_1]^T \quad (13)$$

となり，各行列も

$$A_N = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & -2\zeta_1\omega_1 & 0 & -\omega_1^2 \end{bmatrix}, B_N = \begin{bmatrix} 0 \\ 0 \\ \phi_0 \\ \phi_1 \end{bmatrix} \quad (14)$$

$$C_N = \begin{bmatrix} \phi_0 & \phi_1 & 0 & 0 \\ 0 & 0 & \phi_0 & \phi_1 \end{bmatrix} \quad (15)$$

となる．これらの結果から，ノミナルモデルでの u から $y_1 = \theta$ までの伝達関数 $G_{N1}(s)$ は

$$G_{N1} = \frac{\phi_0^2}{s^2} + \frac{\phi_1^2}{s^2 + 2\zeta_1\omega_1 + \omega_1^2} \quad (16)$$

となる．本稿で用いる数値例として，JAXA が1994年に打上げた ETS-VI(きく6号)の軌道上同定モデルを利用する [2]．フルモデルは $m = 2$ としてロール軸回り面外振動2次モードまでの3つのモードを考慮する．一方，ノミナルモデルは剛体モードと面外1次振動モードの2つのモードを考慮する．Table 1 にフルモデルでの各種パラメータをまとめる．

Table 1: Nominal dynamics model parameters of ETS-VI roll axis motion

mode	natural freq. (ω_j) [Hz]	damping (ζ_j)	mode shape(ϕ_j)
rigid	0	0	0.008
1st bending	0.174	0.005	0.010
2nd bending	0.677	0.005	-0.004

3 深層強化学習による姿勢制御系設計

3.1 深層強化学習の概要

3.1.1 Neural Network

Neural Network (NN) おける関数近似の基本表現を以下のように示す．

$$Q(s_t, a_t) = f(s_t, w) \quad (17)$$

ここで， Q は行動価値関数， f は近似関数， s_t はある時点の状態変数， a_t はある時点の行動， w は重みである．近似関数 f (回帰関数とも呼ばれる) の重みを更新していき最適な f を探索していく，つまりは最適な重みの学習が回帰関数の目的である．基本的な学習プロセスは Fig. 1 に示すマルコフ決定過程に基づいている．

NN の基本的なアルゴリズムを Fig. 2 に示す．Fig. 2 は中間層を1つとし，入力層，中間層，出力層の3層の NN を表している．入力層ニューロンは状態 $s_t = (x, y)$ であり，出力層の $z_1 \sim z_4$ それぞれが行動価値関数に対応している．

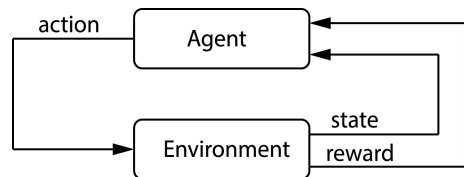


Fig. 1: The agent environment interaction[3].

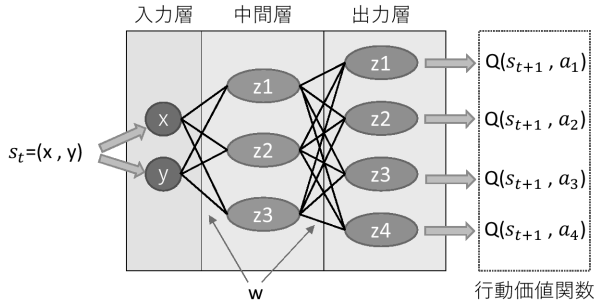


Fig. 2: Neural Network Algorithm.

3.1.2 TD-Q 学習法における NN の行動価値関数回帰

状態 s_t から Q に基づき、次の行動 a_{t+1} が決定され、状態 s_t と行動 a_t から次の状態と行動、 s_{t+1} , a_{t+1} と報酬、 r が算出される。 t を変化させていくたびに算出される行動価値関数 (Q 値) のうち、その最大値 $\max_{a=a_1, a_2, a_3, a_4} Q(s, a)$ が得られる。ここで、最適な w を更新するために以下のような誤差 δ を設定する。

$$\delta = r + \gamma \times \max_{a=a_1, a_2, a_3, a_4} Q_w(s_{t+1}, a) - Q_w(s_t, a) \quad (18)$$

Q_w は、 w によって更新される行動価値関数であり、 δ に基づいて、NN の分野でよく知られる誤差逆伝播法を用いて各層に誤差を伝搬し、 w を更新することができる。なお、 w の更新の式は重みの変化量 Δw を用いて、

$$w \leftarrow w + \Delta w \quad (19)$$

と表される。ここで Δw は学習率 α を用いて、

$$\Delta w = -\alpha \frac{\partial (\frac{1}{2}\delta^2)}{\partial w} \quad (20)$$

と表される。 w によって更新される Q 値を用いた学習手法を Q -learning と呼び、エピソードの終了を待たず、各ステップで更新する手法を Temporal Difference (TD) - Q 法と呼ぶ。

3.1.3 Deep Q-learning

TD-Q 法から拡張し、ディープラーニングと Q -learning を組み合わせた学習手法である Deep Q-Network (DQN) を用いた学習手法、Deep Q-learning が Mnih ら [4] により提案されて以来、高度な学習精度から注目を集めている。DQN にはサンプルデータを生成する NN (Search-NN) が別に備えられており、生成したサンプルデータを格納した経験バッファからランダムに 1 つ以上のデータを取り出し、取り出したデータから誤差 δ を算出する。DQN をはじめとする深層学習を用いて強化学習を行う手法を深層強化学習と呼ぶ。

3.2 MATLAB® の Reinforcement Learning Toolbox™ による設計例

このツールボックスでは、以下のような 4 つのプログラム・関数を準備すると、強化学習における最適解を探索できる。

- 1) メインプログラム (パラメータ設定、学習実行、結果出力)
- 2) ハンドルクラス定義関数 (変数定義、力学モデル・離散型運動方程式の定義)
- 3) リセット関数 (変数の初期化、変動パラメータの初期化)
- 4) ステップ関数 (1 アクション実行後の報酬定義)

本稿ではこのツールボックスの使い方を概説する。まず、1) メインプログラムでは、制御モデルをハンドルクラス構造体として以下のように定義する。

```
%制御モデルを構造体として定義
Sys_Mdl = class_Sys_Mdl(Ad,Bd,C,x0,...
Ts,zeta1,zeta2,omg1,omg2,phi0,phi1,phi2,f0,ygain);
```

次に観測量とアクション (制御トルクに相当) の型を定義する。

```
%観測する状態量のサイズと上下限を定義
%観測量は姿勢角と姿勢角速度の 2 つ、上下限は不要
ObservationInfo = rlNumericSpec([2,1]);...
%LowerLimit', [-10;-10], 'UpperLimit', [10;10]);
```

```
%行動の定義
%制御トルクは{0, +/-0.5, +/-1}の
%5 通りを行動として離散型で定義
ActionInfo = rlFiniteSetSpec([-1:0.5:1]);
```

今回の例では、アクション数を 5 通りに設定し、制御トルクは最大 0.2[Nm] の制約があるため、 $f0=0.02$ として、最大トルク $f0$ を係数としてアクションに掛けて制御トルクとしている。そしてリセット関数とステップ関数名を定義する。

```
%リセット関数、ステップ関数の定義
ResetHandle = @() Sys_Mdl_ResetFunc(Sys_Mdl);
StepHandle = @(Action, LoggedSignals) Sys_Mdl_StepFunc...
(LoggedSignals, Action, Sys_Mdl);
```

最後に環境とエージェントを定義して、学習を実行する。ここでは 1 エピソードにおける計算ステップ数は 300 回とした。

```
%環境、エージェントの定義
env = rlFunctionEnv(ObservationInfo, ActionInfo,...
StepHandle, ResetHandle);
agent = rlDQNAgent(ObservationInfo, ActionInfo);
%学習の実行
trainStats = train(agent, env, opt);
```

3) リセット関数においては、毎回のエピソード開始時にパラメータ変動を起こして、学習させることで、設計された制御器のロバスト性を担保する。その具体的なコードはリセット関数の冒頭で

```

omg1=Sys_Mdl.omg10*(1+2*(rand-0.5)*0.02);
omg2=Sys_Mdl.omg20*(1+2*(rand-0.5)*0.02);
phi0=Sys_Mdl.phi00*(1+2*(rand-0.5)*0.02);
phi1=Sys_Mdl.phi10*(1+2*(rand-0.5)*0.02);
phi2=Sys_Mdl.phi20*(1+2*(rand-0.5)*0.02);
%_/-2%変動
zeta1=Sys_Mdl.zeta10*(1-(rand)*0.20);
zeta2=Sys_Mdl.zeta20*(1-(rand)*0.20);
%_0/-20%変動

```

と宣言する。最後に 4) ステップ関数においては、1 アクション実行後の状態量を用いて報酬式を定義する。その具体的なコードは

```

%_報酬の計算
Energy=_15*abs(y(1))+5*abs(y(2));
Reward=_-20*Energy-200*abs(Action);

```

とした。Energy は姿勢角、姿勢角速度が目標値に近づくほどに小さくなるよう設計されており、報酬式 Reward は Energy と入力量 Action が小さいほど、高い報酬を得られるようになっている。すなわち、姿勢角、姿勢角速度が目標値に近づくほどに、入力量が小さいほどに高い報酬が得られる。このような手法で強化学習によりロバストな姿勢制御系を学習させた結果を以下に示す。Fig. 3 に学習曲線を、また Fig. 4 に得られたニューラルネットを用いて 0.05 度のステップ応答をさせた場合の姿勢角、姿勢角速度、制御トルクの移管履歴を示す。学習曲線は青線に各エピソードの報酬和、水色線に直近 50 回の報酬和の平均、黄色線にエピソード開始時の Q 値 (Q_0) を表す。エピソード数が増えるごとに高い報酬和が得られている。 Q 値は学習が進むにつれて最適な Q 値に収束するわけであるが、収束には至っていないものの、強化学習の特性として Q 値の収束前においても学習の途中になるべく多くの報酬を得るようになる傾向があり [5]、高い報酬を得られる制御器を設計できた時点で学習を終了した。今回の学習結果では、122 回のエピソード学習後、ほぼバンバン解に近い応答波形となっている。整定後は制御トルクの無駄打ちもほとんどなく、理想的な波形が得られた。

3.3 他の AI モデル (DQN) による設計例

3.2 節で MATLAB[®] Reinforcement Learning Toolbox[™] の適応を示したが、Toolbox としての制約があることから、より自由度の高い学習アルゴリズムでの設計を目指すため、曾我部が構築した倒立振り課題 (cart pole) 用の DQN[6] を改変したアルゴリズムで以後の制御系設計を行う。

3.2 節と異なり、5 通りのアクション (制御トルク) を離散型でなく、連続型として設計してみる。ここで言う連続型とは、アクションで与えるのが微小な変異量となっており、ある時点での制御トルク量に微小な変化を与えることによって連続的な入力に近い波形が

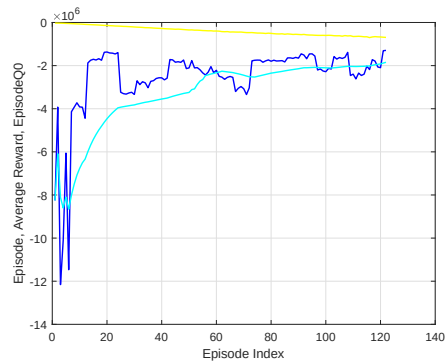


Fig. 3: Episode reward for rlFunctionEnv with rlDQ-NAgent (average episode: light blue, episode: dark blue, Q_0 : yellow).

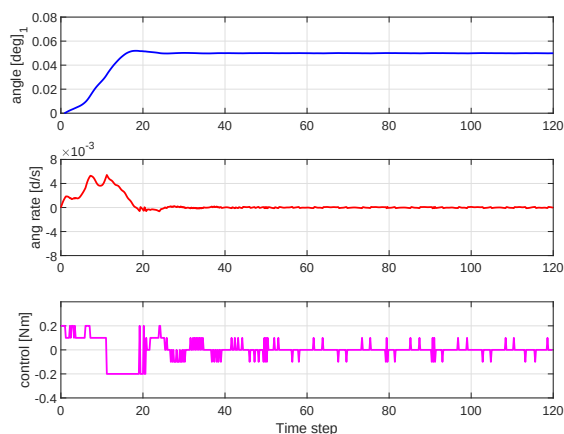


Fig. 4: 0.05 [deg] step response by discrete input controller (upper: attitude [deg], middle: attitude rate [deg/s], lower: control torque [Nm]).

得られるというものである。3.2 節との大きな違いは、観測量とアクションである。観測量は姿勢角と姿勢角速度としていたが、連続型ではある時点での制御トルク量も必要となる。ここで、アクションについては前ステップに微小変異量を加えるものとし、

```

Action=_[Action-0.01*_Action+0.01
          Action-0.1*_Action+0.1*_Action];

```

とした。また、報酬設計については

```

%_報酬の設定
Energy=_3*abs(y(1))+1*abs(y(2));
Reward=_-20*Energy-0.1*abs(Action);

```

とした。使用した AI モデルの違いから、試行錯誤的に設定した報酬式の重みに違いがあるものの、基本的な構造は 3.2 節で用いた報酬式と同一である。3.2 節と同様、動的モデルのパラメータに変動を起こすことで、強化学習によりロバストな姿勢制御系を学習させた結果を以下に示す。Fig. 5 に得られたニューラルネットを用いて 0.05 度のステップ応答をさせた場合の姿勢角、姿勢角速度、制御トルクの移管履歴を示す。今回の学

習結果では、1000回のエピソード学習後、3.2節よりもゆっくりとした応答波形となっている。これは連続型をとったことにより入力の変更にできないために、応答が遅くなってしまったものと考えられる。整定後は制御トルクの無駄打ちもほとんどみられない。

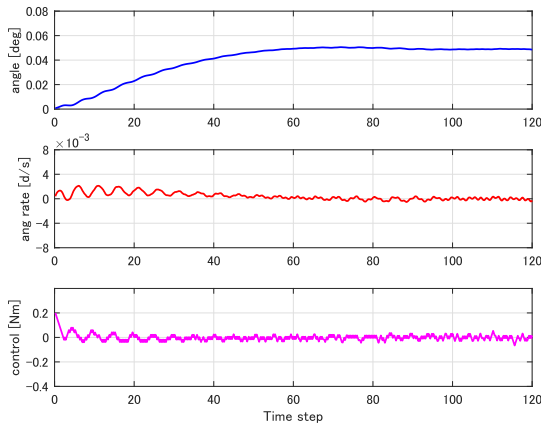


Fig. 5: 0.05 [deg] step response by continuous controller(upper:attitude [deg]. middle:attitude rate [deg/s], lower:control torque [Nm]).

3.4 姿勢角、姿勢角速度の初期値に対するロバスト性

3.2, 3.3節で示した動的モデルのパラメータ変動に関するロバスト性だけでなく、姿勢角、姿勢角速度の初期値についてもロバスト性を有する制御系を設計してみる。これについても3.2, 3.3節同様、ロバスト性を与えたい姿勢角、姿勢角速度の初期値に変動を与えて学習させることで有効な制御系を設計できると考えられる。姿勢角 (-0.05~0.05[deg])、姿勢角速度 (-0.002~0.002[deg/s]) の平衡点 (姿勢角:0.05[deg], 姿勢角速度:0[deg/s]) からの初期値に与えて学習させた結果をFig. 6に示す。これは、姿勢角の初期値を0.08[deg]、姿勢角速度の初期値を-0.002[deg/s]としたときの例である。また、本節で構築したニューラルネットワークと初期値を固定で学習させた3.3節のニューラルネットワークとの比較をFig. 7に示す。青点は3.3節のニューラルネットワークによる制御結果、赤点は本節のニューラルネットワークによる制御結果を表しており、z軸方向は姿勢角の終端誤差を表している。本節で設計したニューラルネットワークのほうが姿勢角、姿勢角速度の初期値に対する高いロバスト性をもつ傾向にあることが期待できる。

3.5 周波数応答特性を用いた観測雑音に対するロバスト性

前節で述べた初期値に対するロバスト性以外に、モデル化誤差に起因する観測雑音に対し、抑制機能をもつ

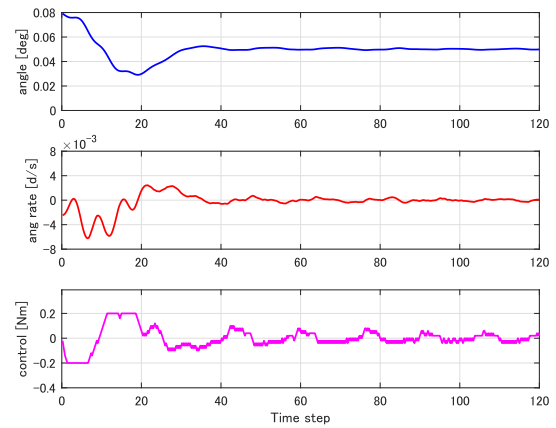


Fig. 6: 0.03 [deg] step response by DQN controller(upper:attitude [deg]. middle:attitude rate [deg/s], lower:control torque [Nm]).

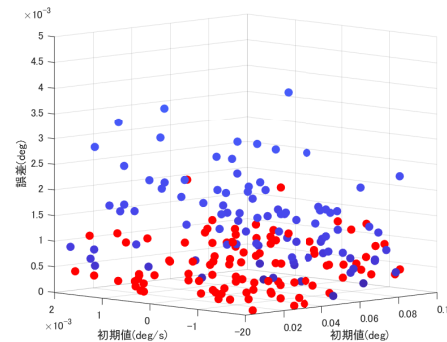


Fig. 7: Termination error for random initial values.

たロバスト制御器を強化学習により設計する。外乱や観測雑音を抑制可能な制御器の特性として、広帯域の観測雑音に対してもその感度を下げられることが挙げられる。Fig. 10に示すブロック線図における外乱 w に人為的な観測雑音を制御器に印加して、 $K(s)$ の代わりにニューラルネット型観測雑音抑制制御器を強化学習により設計する。報酬設計は、基本的に3.2節と同様であるが、整定後、フーリエ変換が有効となるデータ量が集まってからの報酬を以下のようにした。

%%報酬の設定

```
Energy□=□15*abs(y(1))+5*abs(y(2));
```

```
Reward□=□(-20*Energy-0.1*abs(Action))*Mp/Mn;
```

この報酬式のうち、3.2節と異なるのは報酬式全体を (Mp/Mn) 倍している点である。ここで Mn は観測雑音として印加したランダムな周波数を有する正弦波の周波数領域でのパワーを、 Mp はプラント出力(ここでは姿勢角に相当)の周波数領域でのその正弦波周波数におけるパワーを表している。つまり、 (Mp/Mn) は観測雑音に対する制御器を含めたプラントの感度を意味している。よってここでの報酬式は、観測雑音として

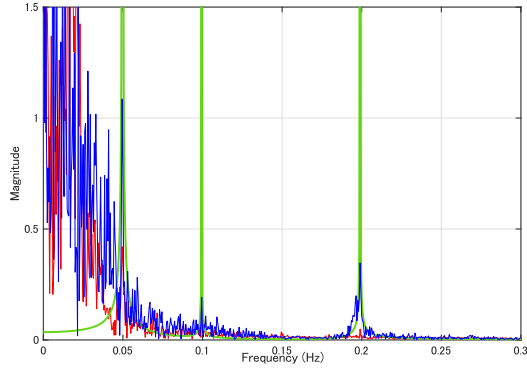


Fig. 8: Disturbance suppression for sinusoidal noise.

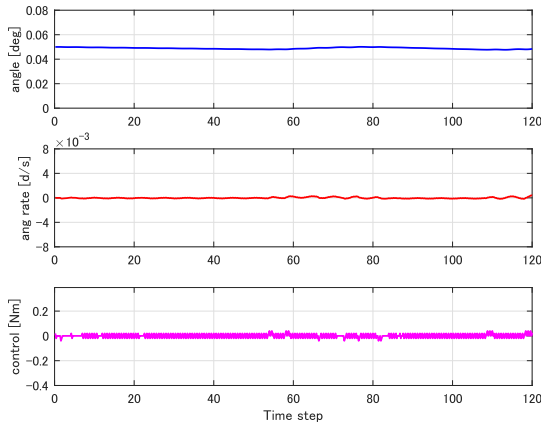


Fig. 9: 0.05 [deg] step response by disturbance suppression controller(upper:attitude [deg], middle:attitude rate [deg/s], lower:control torque [Nm]).

加えた正弦波を抑えるほどに大きな報酬が得られる設計になっている。

以上の設計手法の有効性を調べるため、観測雑音として振幅が $5e-4$ [deg], 周波数が 0.05 [Hz], 0.1 [Hz], 0.2 [Hz] の 3 種類の正弦波を合成した観測雑音を観測量に印加させたとき、プラントの姿勢角がどの程度の影響を受けるかを、周波数領域での姿勢角応答信号のパワーを調べることでロバスト性を評価した。Fig. 8 に、本節で設計した外乱抑圧コントローラ（赤線）と 3.3 節で設計したコントローラ（青線）を用いた際の周波数応答を示す。緑線は印加した 3 種の正弦波を合成した観測雑音である。この際用いたパラメータは Table 1 と同様である。3.3 節のコントローラに比べ、本節の周波数応答特性を用いた外乱抑制制御器の方が、印加された周波数帯での振動が少なく、観測雑音に対して、よりロバストな性質を持つことが確認された。

また、3 種の正弦波を合成した観測雑音が印加された場合のステップ応答を Fig. 9 に示す。姿勢角、姿勢角速度に振動が見受けられるものの、整定を保っている。

4 H_∞ 混合感度問題による姿勢制御系設計

柔軟構造衛星のロバスト制御系設計の最も代表的な手法は LMI による混合感度問題での設計と考えられる。ここでは、制御対象は剛体モード + 1 次振動モードによるモデルとして、Fig. 10 に示すようなブロック線図を想定して、混合感度問題： w から $z = [z_1 \ z_2]^T$ までの伝達関数の H_∞ ノルム：

$$\|G_{zw}\|_\infty = \left\| \begin{bmatrix} W_S(s)S(s) \\ W_T(s)T(s) \end{bmatrix} \right\|_\infty < \gamma \quad (21)$$

を満足する解を求めてみる。ここで $S(s)$ は感度関数、 $T(s)$ は相補感度関数である。混合感度問題ではまず感

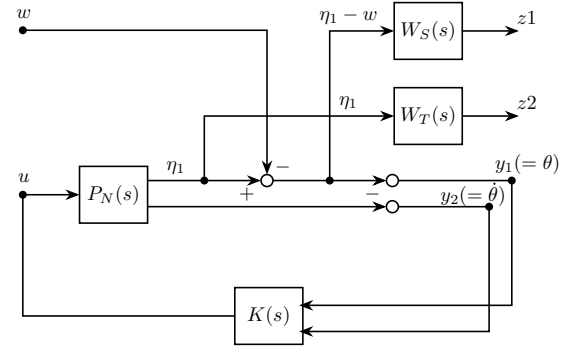


Fig. 10: Block diagram of typical mixed sensitivity design

度重み $W_S(s)$ と相補感度重み $W_T(s)$ を設計するため、プラントの乗法的変動を計算する必要がある。モデル化誤差やパラメータ変動に伴う乗法的変動は、固有角周波数、モード減衰比、モード形状値のパラメータ変動分をそれぞれ $\delta\omega_i$, $\delta\zeta_i$, $\delta\phi_i$ とすると

$$\Delta_m(s) = P_N^{-1}P_F - I = P_N^{-1}(P_F - P_N) \quad (22)$$

$$= \frac{2\delta\phi_0 + \delta\phi_0^2}{\phi_0^2} \quad (23)$$

$$+ \sum_1^2 \frac{(\phi_i + \delta\phi_i)^2 / \phi_0^2}{s^2 + 2(\zeta_i + \delta\zeta_i)(\omega_i + \delta\omega_i) + (\omega_i + \delta\omega_i)^2}$$

となる。

4.1 感度・相補感度関数重みの設計

感度関数重みは剛体モードの伝達関数を参考にして、原点 2 重極を移動させた

$$W_S(s) = \frac{a_S\phi_0^2}{s^2 + b_Ss + c_S} \quad (24)$$

のような 2 次ローパス型フィルタ形式とした。この重み関数は Fig. 10 におけるスカラー y_1 からスカラー z_1 への伝達関数となるので

$$z_1 = \frac{a_S\phi_0^2}{s^2 + b_Ss + c_S} y_1 \quad (25)$$

であり、これを内部状態変数 $\mathbf{x}_S$ を用いて、状態空間表現すると

$$\dot{\mathbf{x}}_S = \underbrace{\begin{bmatrix} 0 & 1 \\ -c_S & -b_S \end{bmatrix}}_{=A_S} \mathbf{x}_S + \underbrace{\begin{bmatrix} 0 \\ \phi_0 \end{bmatrix}}_{=B_S} y_1, \quad z_1 = \underbrace{a_S[\phi_0 \ 0]}_{=C_S} \mathbf{x}_S \quad (26)$$

となる．(25) 式の型にすることで、低周波帯域では $\frac{a_S \phi_0^2}{c_S}$ で一定値になり、高周波帯域では $1/s^2$ の傾きを持つ右肩下りの特性を有することになる．

一方、相補感度関数重みは残余モデルのボード線図から、 $\Delta_m(s)$ のゲイン曲線を包絡するようなハイパス型フィルタとし、ここでは双2次進み補償フィルタ形式

$$W_T(s) = \frac{a_T s^2}{s^2 + b_T s + c_T} \quad (27)$$

とした．この重み関数は Fig. 10 におけるスカラー η_1 からスカラー z_2 への伝達関数となるので

$$z_2 = \frac{a_T s^2}{s^2 + b_T s + c_T} \eta_1 \quad (28)$$

であり、これを内部状態変数 $\mathbf{x}_T$ を用いて、状態空間表現すると

$$\dot{\mathbf{x}}_T = \underbrace{\begin{bmatrix} 0 & 1 \\ -c_T & -b_T \end{bmatrix}}_{=A_T} \mathbf{x}_T + \underbrace{\begin{bmatrix} 0 \\ a_T \end{bmatrix}}_{=B_T} \eta_1 \quad (29)$$

$$z_2 = \underbrace{\begin{bmatrix} -c_T & -b_T \end{bmatrix}}_{=C_T} \mathbf{x}_T + \underbrace{a_T}_{=D_T} \eta_1 \quad (30)$$

となる．Table 2 に設計結果を示す．なお、(21) 式の γ はここでは最適化せず、事前に定数とした．Fig. 11 に乗法的変動と設計された相補感度重みのボード線図を示す．

Table 2: Parameters of weighting functions of sensitivity and complementary sensitivity functions.

func.	param1	param2	param3
$W_S(s)$	$a_S = 25$	$b_S = 0.05$	$c_S = 1e-7$
$W_T(s)$	$a_T = 14$	$b_T = 0.8$	$c_T = 1.1$
$H_\infty \text{norm}$	$\gamma = 0.85$		

4.2 混合感度設計での 0.05 度のステップ応答例

以上のように混合感度問題で設計されたロバスト制御系 (A_C 行列の次数は 8 次) を組み込んで 0.05 度のステップ応答の数値シミュレーションを 120 秒間、実行した結果を Fig. 12 に示す．強化学習の結果と比べて、

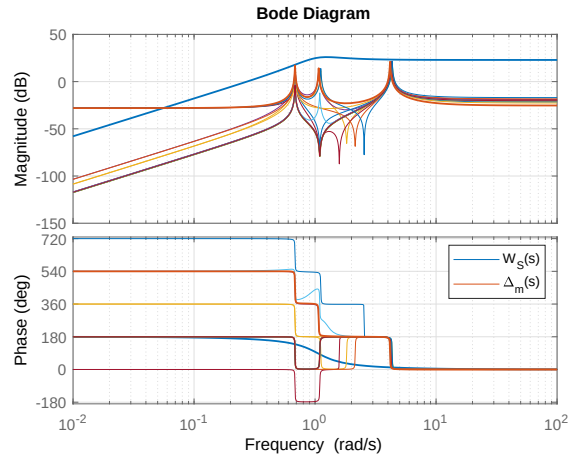


Fig. 11: Bode diagram of $\Delta_M(s)$ and $W_T(s)$.

制御トルクの出力が抑えられているため、ゆっくりした応答波形になっている．これは LMI 解法時に、リッカチ行列の片方を最小化して、できるだけ出力ゲインを抑えているためである．1994 年当時に実施された軌道上実験でも、制御トルクの飽和を避けるため、低ゲインでの制御系実装となっていた [7]．

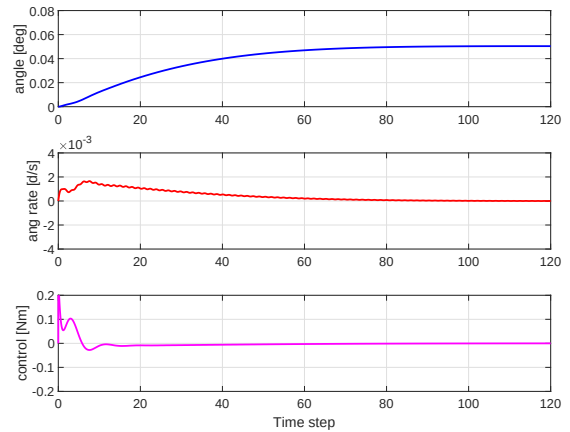


Fig. 12: Step response of robust attitude control system via mixed sensitivity design.

5 構造指定制御器による姿勢制御系設計

プラントのパラメータが変動する今回の問題設定では、構造化特異値による μ 解析手法の適用も可能であるが、一般に μ 解析では設計される制御器の次数が高次になる傾向があり、実装上の問題を抱えている．そこで 2010 年以降、制御器の構造を指定できるノンスムーズソルバーが考案されている．そこで、ここでは hinfstruct コマンドを利用してこの H_∞ 混合感度問題に応用してみたい [8]．

4 節と同様に、不確かさを有するパラメータについてのロバスト性を検証するため、パラメータ変動を意図

的に生起させ、応答性を評価する。この際、制御器もその構造を予め指定できるので、本稿では、チューナブル PD 制御器 + 2 次のリードラグフィルタとした。ここでチューナブル PD 制御器で用いたゲインは、 $K_p = 5$ 、 $K_d = 500$ である。また、感度・相補感度重みは 4 節と同様の値を使用した。

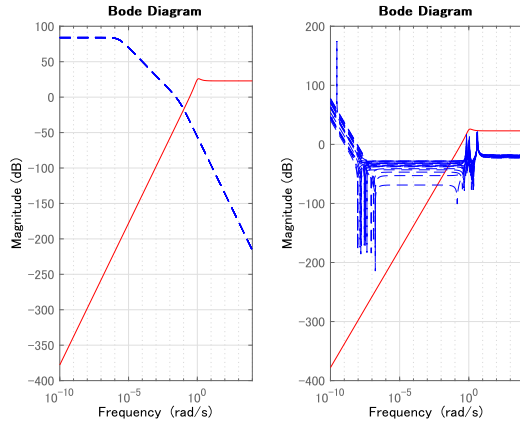


Fig. 13: Bode diagram of $W_S(s)$, $\Delta_M(s)$ and $W_T(s)$.

Fig. 13 に設計された重み関数のボード線図を示す。

29 回の繰り返し計算後、最適 H_∞ ノルム $\gamma = 0.7149$ として得られた制御器による 0.05 度のステップ応答を Fig. 14 に示す。

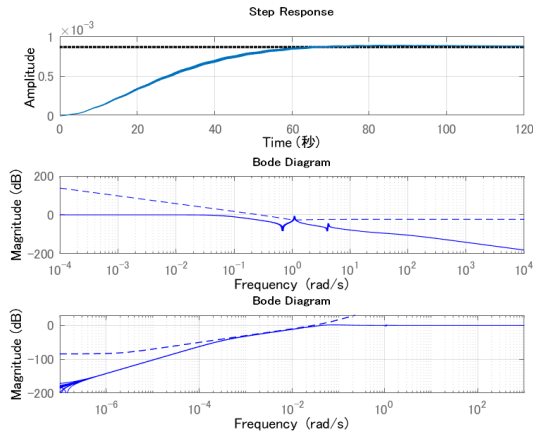


Fig. 14: Step response of robust attitude control system via "hinfstruct" mixed sensitivity design.

設計すべき制御器の構造をこのように PD + 4 次フィルタに固定しても、十分なロバスト性能を有する制御器の設計が可能であり、ステップ応答の波形はオーバーシュートもほとんどなく、次数の小さな制御器でも満足のいく結果となっていることが確認できる。

6 おわりに

本稿では、柔軟構造衛星のロバスト姿勢制御系設計を深層強化学習によるニューラルネットで実現する方法について検討し、従来法である混合感度問題での解や制御器構造を固定した最近の手法と比較をした。ステップ応答による制御器性能の比較では、ほぼ同様の性能が確認でき、いずれの方法においても有効な制御が可能であるということが示せた。

深層強化学習を用いたロバスト制御系設計においては、姿勢角、姿勢角速度の初期値をランダムに設定することで、初期値の微小変動に対するロバスト性の向上を図ることができた。また、学習にランダムな正弦波外乱を用いることにより、観測雑音抑制制御器を設計できた。しかしながら、本稿で用いた深層強化学習法では、ニューラルネットワークを用いた非線形な制御器を用いているため、理論的な安定性の保証が困難である。今後は、統計的手法等により実用化の可能性を検討していく。

参考文献

- [1] P.W. Likins: *Dynamics and Control of Flexible Space Vehicles*, Technical Report 32-1329, JPL (1970)
- [2] 航空宇宙技術研究所, 東芝: 大型柔軟構造衛星の同定制御に関する研究 共同研究成果報告書, (1997)
- [3] Changxi You, Jianbo Lu, Dimitar Filev, Panagiotis Tsiotras, *Advanced Planning for Autonomous Vehicles Using Reinforcement Learning and Deep Inverse Reinforcement Learning*, Robotics and Autonomous Systems, Vol.114, pp. 1-18 (2019)
- [4] Mnih, V., et. al., *Human-Level Control Through Deep Reinforcement Learning*, Nature, Vol.518, pp. 529-533 (2015)
- [5] 木村元, 宮崎和光, 小林重信: 強化学習システムの設計指針; 計測と制御, Vol. 38, No. 10, pp. 618-623 (1999)
- [6] 曾我部東馬, 強化学習アルゴリズム入門, オーム社, pp. 143-152 (2019)
- [7] 木田, 山口, 千田, 関口: H_∞ 制御による ETS-VI の軌道上での姿勢制御実験; 日本航空宇宙学会誌, Vol. 45, No. 520, pp. 284-294 (1997)
- [8] 赤坂: Robust Control Toolbox による構造が固定された制御系の解析・設計; 計測と制御, Vol.59, No.3, pp. 160-168 (2020)